

Copy_of_mini_project_diffusion_F25

December 4, 2025

1 Mini Project: Diffusion

This mini project is [adapted](#) from UC Berkeley CS180 with permission.

Please create a copy of this notebook before working on it.

In this problem set you will play around with diffusion models, implement diffusion sampling loops, and use them for interesting applications such as creating optical illusions.

2 Part 0: Setup

2.0.1 Using DeepFloyd

We are going to use the [DeepFloyd IF](#) diffusion model. DeepFloyd is a two stage model trained by Stability AI. The first stage produces images of size 64×64 and the second stage takes the outputs of the first stage and generates images of size 256×256 .

Before using DeepFloyd, you must accept its usage conditions. To do so:

1. Make a [Hugging Face account](#) and log in.
2. Accept the license on the model card of [DeepFloyd/IF-I-XL-v1.0](#). Accepting the license on the stage I model card will auto accept for the other IF models.
3. Log in locally by entering your [Hugging Face Hub access token](#) below. You should be able to find and create tokens [here](#).

```
[ ]: from huggingface_hub import login

token = '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'
login(token=token)
```

2.0.2 Install Dependencies

Run the below to install dependencies.

```
[ ]: ! pip install -q \
diffusers \
transformers \
safetensors \
sentencepiece \
accelerate \
```

```
bitsandbytes \
einops \
mediapy \
accelerate
```

2.0.3 Import Dependencies

The cell below imports useful packages we will need, and setup the device that we're using.

```
[ ]: from PIL import Image
import mediapy as media
from pprint import pprint
from tqdm import tqdm

import torch
import torchvision.transforms.functional as TF
import torchvision.transforms as transforms
from diffusers import DiffusionPipeline
from transformers import T5EncoderModel

# For downloading web images
import requests
from io import BytesIO

device = 'cuda'
```

2.0.4 Loading the models

We will need to download and create the two DeepFloyd stages. These models are quite large, so this cell may take a minute or two to run.

```
[ ]: # Load DeepFloyd IF stage I
stage_1 = DiffusionPipeline.from_pretrained(
    "DeepFloyd/IF-I-L-v1.0",
    text_encoder=None,
    variant="fp16",
    torch_dtype=torch.float16,
)
stage_1.to(device)

# Load DeepFloyd IF stage II
stage_2 = DiffusionPipeline.from_pretrained(
    "DeepFloyd/IF-II-L-v1.0",
    text_encoder=None,
    variant="fp16",
    torch_dtype=torch.float16,
)
stage_2.to(device)
```

```

A mixture of fp16 and non-fp16 filenames will be loaded.
Loaded fp16 filenames:
[text_encoder/pytorch_model.bin.index.fp16.json,
safety_checker/model.fp16.safetensors,
text_encoder/model.fp16-00001-of-00002.safetensors,
unet/diffusion_pytorch_model.fp16.safetensors,
text_encoder/model.fp16-00002-of-00002.safetensors,
text_encoder/model.safetensors.index.fp16.json]
Loaded non-fp16 filenames:
[watermarker/diffusion_pytorch_model.safetensors]
If this behavior is not expected, please check your folder structure.

Loading pipeline components...:  0%|          | 0/6 [00:00<?, ?it/s]

```

```

A mixture of fp16 and non-fp16 filenames will be loaded.
Loaded fp16 filenames:
[text_encoder/pytorch_model.bin.index.fp16.json,
safety_checker/model.fp16.safetensors,
text_encoder/model.fp16-00001-of-00002.safetensors,
unet/diffusion_pytorch_model.fp16.safetensors,
text_encoder/model.fp16-00002-of-00002.safetensors,
text_encoder/model.safetensors.index.fp16.json]
Loaded non-fp16 filenames:
[watermarker/diffusion_pytorch_model.safetensors]
If this behavior is not expected, please check your folder structure.

Loading pipeline components...:  0%|          | 0/7 [00:00<?, ?it/s]

```

```

[ ]: IFSuperResolutionPipeline {
  "_class_name": "IFSuperResolutionPipeline",
  "_diffusers_version": "0.35.2",
  "_name_or_path": "DeepFloyd/IF-II-L-v1.0",
  "feature_extractor": [
    "transformers",
    "CLIPImageProcessor"
  ],
  "image_noising_scheduler": [
    "diffusers",
    "DDPMScheduler"
  ],
  "requires_safety_checker": true,
  "safety_checker": [
    "deepfloyd_if",
    "IFSafetyChecker"
  ],
  "scheduler": [
    "diffusers",

```

```

        "DDPMScheduler"
    ],
    "text_encoder": [
        null,
        null
    ],
    "tokenizer": [
        "transformers",
        "T5Tokenizer"
    ],
    "unet": [
        "diffusers",
        "UNet2DConditionModel"
    ],
    "watermarker": [
        "deepfloyd_if",
        "IFWatermarker"
    ]
}

```

2.0.5 Disclaimer about Text Embeddings

DeepFloyd was trained as a text-to-image model, which takes text prompts as input and outputs images that are aligned with the text. Throughout this notebook, you will see that we ask you to generate with the prompt “a high quality photo”. We want you to think of this as a “null” prompt that doesn’t have any specific meaning, and is simply a way for the model to do unconditional generation. You can view this as using the diffusion model to “force” a noisy image onto the “manifold” of real images.

In the later sections, we will guide this project with a more detailed text prompt.

2.0.6 Downloading Precomputed Text Embeddings

Because the text encoder is very large, and barely fits on a free tier Colab GPU, we have precomputed a couple of text embeddings for you to try. This should hopefully save some headaches from GPU out of memory errors. At the end of the homework, we provide you code if you want to try your own text prompts. If you’d like, you can pay for Colab Pro and avoid needing to load the two models on different sessions.

```

[ ]: !wget https://cal-cs180.github.io/fa24/hw/proj5/prompt_embeds_dict.pth -O
      ↪ prompt_embeds_dict.pth

prompt_embeds_dict = torch.load('prompt_embeds_dict.pth')

pprint(list(prompt_embeds_dict.keys()))

```

```

--2025-12-04 07:36:24--  https://cal-
cs180.github.io/fa24/hw/proj5/prompt_embeds_dict.pth
Resolving cal-cs180.github.io (cal-cs180.github.io)... 185.199.108.153,

```

```
185.199.109.153, 185.199.110.153, ...
Connecting to cal-cs180.github.io (cal-cs180.github.io)|185.199.108.153|:443...
connected.
HTTP request sent, awaiting response... 200 OK
Length: 8836210 (8.4M) [application/octet-stream]
Saving to: 'prompt_embeds_dict.pth'
```

```
prompt_embeds_dict. 100%[=====>]    8.43M  --.-KB/s    in 0.04s
```

```
2025-12-04 07:36:25 (201 MB/s) - 'prompt_embeds_dict.pth' saved
[8836210/8836210]
```

```
['an oil painting of a snowy mountain village',
 'a photo of the amalfi cost',
 'a photo of a man',
 'a photo of a hipster barista',
 'a photo of a dog',
 'an oil painting of people around a campfire',
 'an oil painting of an old man',
 'a lithograph of waterfalls',
 'a lithograph of a skull',
 'a man wearing a hat',
 'a high quality photo',
 '',
 'a rocket ship',
 'a pencil']
```

2.0.7 Seed your Work

To reproduce your code, please use a random seed from this point onward.

```
[ ]: def seed_everything(seed):
      torch.cuda.manual_seed(seed)
      torch.manual_seed(seed)

YOUR_SEED = 180
seed_everything(YOUR_SEED)
```

2.0.8 Sampling from the Model

The objects instantiated above, `stage_1` and `stage_2`, already contain code to allow us to sample images using these models. Read the code below carefully (including the comments) and then run the cell to generate some images. Play around with different prompts and `num_inference_steps`.

2.0.9 Deliverables

- For the 3 text prompts that we provide, display the caption and the output of the model. Briefly reflect on the quality of the outputs and their relationships to the text prompts. Try

a different `num_inference_steps` for at least one of the images, showing before and after. Make sure to try at least 2 different `num_inference_steps` values.

- Report the random seed you used. You should use the same seed for all subsequent parts.

```
[ ]: # Get prompt embeddings from the precomputed cache.
# `prompt_embeds` is of shape [N, 77, 4096]
# 77 comes from the max sequence length that deepfloyd will take
# and 4096 comes from the embedding dimension of the text encoder
# `negative_prompt_embeds` is the same shape as `prompt_embeds` and is used
# for Classifier Free Guidance. You can find out more from:
# - https://arxiv.org/abs/2207.12598
# - https://sander.ai/2022/05/26/guidance.html
prompts = [
    'an oil painting of a snowy mountain village',
    'a man wearing a hat',
    "a rocket ship",
]
prompt_embeds = torch.cat([
    prompt_embeds_dict[prompt] for prompt in prompts
], dim=0)
negative_prompt_embeds = torch.cat(
    [prompt_embeds_dict['']] * len(prompts)
)

stage_1_output_hq = stage_1(
    prompt_embeds=prompt_embeds,
    negative_prompt_embeds=negative_prompt_embeds,
    num_inference_steps=20,
    output_type="pt"
).images
stage_2_output_hq = stage_2(
    image=stage_1_output_hq,
    num_inference_steps=20,
    prompt_embeds=prompt_embeds,
    negative_prompt_embeds=negative_prompt_embeds,
    output_type="pt",
).images

# Sample from stage 1
# Outputs a [N, 3, 64, 64] torch tensor
# num_inference_steps is an integer between 1 and 1000, indicating how many
# denoising steps to take: lower is faster, at the cost of reduced quality
stage_1_output = stage_1(
    prompt_embeds=prompt_embeds,
    negative_prompt_embeds=negative_prompt_embeds,
    num_inference_steps=5,
    output_type="pt"
```

```

).images

# Sample from stage 2
# Outputs a [N, 3, 256, 256] torch tensor
# num_inference_steps is an integer between 1 and 1000, indicating how many
# denoising steps to take: lower is faster, at the cost of reduced quality
stage_2_output = stage_2(
    image=stage_1_output,
    num_inference_steps=5,
    prompt_embeds=prompt_embeds,
    negative_prompt_embeds=negative_prompt_embeds,
    output_type="pt",
).images

# Display images
# We need to permute the dimensions because `media.show_images` expects
# a tensor of shape [N, H, W, C], but the above stages gives us tensors of
# shape [N, C, H, W]. We also need to normalize from [-1, 1], which is the
# output of the above stages, to [0, 1]
media.show_images(
    stage_1_output.permute(0, 2, 3, 1).cpu() / 2. + 0.5,
    titles=prompts)
media.show_images(
    stage_2_output.permute(0, 2, 3, 1).cpu() / 2. + 0.5,
    titles=prompts)
media.show_images(
    stage_1_output_hq.permute(0, 2, 3, 1).cpu() / 2. + 0.5,
    titles=prompts)
media.show_images(
    stage_2_output_hq.permute(0, 2, 3, 1).cpu() / 2. + 0.5,
    titles=prompts)

comments = """
When we reduce the num_inference_steps, we can see that the noise in the image_
↳ is clearly not fully removed.
"""
print(comments)

```

```
0%|          | 0/20 [00:00<?, ?it/s]
```

```
0%|          | 0/20 [00:00<?, ?it/s]
```

```
0%|          | 0/5 [00:00<?, ?it/s]
```

```
0%|          | 0/5 [00:00<?, ?it/s]
```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

When we reduce the `num_inference_steps`, we can see that the noise in the image is clearly not fully removed.

3 Part 1: Sampling Loops

In this part of the problem set, you will write your own “sampling loops” that use the pretrained DeepFloyd denoisers. These should produce high quality images such as the ones generated above.

You will then modify these sampling loops to solve different tasks such as producing optical illusions.

3.0.1 Diffusion Models Primer

([Image Source](#))

Starting with a clean image, x_0 , we can iteratively add noise to an image, obtaining progressively more and more noisy versions of the image, x_t , until we’re left with basically pure noise at timestep $t = T$. When $t = 0$, we have a clean image, and for larger t more noise is in the image.

A diffusion model tries to reverse this process by denoising the image. By giving a diffusion model a noisy x_t and the timestep t , the model predicts the noise in the image. With the predicted noise, we can either completely remove the noise from the image, to obtain an estimate of x_0 , or we can remove just a portion of the noise, obtaining an estimate of x_{t-1} , with slightly less noise.

To generate images from the diffusion model (sampling), we start with pure noise at timestep T sampled from a gaussian distribution, which we denote x_T . We can then predict and remove part of the noise, giving us x_{T-1} . Repeating this process until we arrive at x_0 gives us a clean image.

For the DeepFloyd models, $T = 1000$.

3.0.2 Setup

The exact amount of noise added at each step is dictated by noise coefficients, $\bar{\alpha}_t$, which were chosen by the people who trained DeepFloyd. Run the cell below to create `alphas_cumprod`, which retrieves these coefficients and downloads a test image that we will work with.

```
[ ]: # Get scheduler parameters
      alphas_cumprod = stage_1.scheduler.alphas_cumprod

      # Get test image
      !wget cal-cs180.github.io/fa24/hw/proj5/assets/campanile.jpg -O campanile.jpg
      test_im = Image.open('campanile.jpg')

      # For stage 1: Resize to (64, 64), convert to tensor, rescale to [-1, 1], and
      # add a batch dimension. The result is a (1, 3, 64, 64) tensor
```



```

test_im = Image.open('campanile.jpg').resize((64, 64))
test_im = TF.to_tensor(test_im)
test_im = 2 * test_im - 1
test_im = test_im[None]

# Show test image
print('Test image:')
media.show_image(test_im[0].permute(1,2,0) / 2. + 0.5)

```

```

URL transformed to HTTPS due to an HSTS policy
--2025-12-04 07:37:09--  https://cal-
cs180.github.io/fa24/hw/proj5/assets/campanile.jpg
Resolving cal-cs180.github.io (cal-cs180.github.io)... 185.199.108.153,
185.199.109.153, 185.199.110.153, ...
Connecting to cal-cs180.github.io (cal-cs180.github.io)|185.199.108.153|:443...
connected.
HTTP request sent, awaiting response... 200 OK
Length: 75608 (74K) [image/jpeg]
Saving to: 'campanile.jpg'

```

```

campanile.jpg      100%[=====>]  73.84K  --.-KB/s    in 0.001s

```

```

2025-12-04 07:37:09 (51.3 MB/s) - 'campanile.jpg' saved [75608/75608]

```

Test image:

<IPython.core.display.HTML object>

4 1.1 Implementing the forward process

Disclaimer about equations: Colab cannot correctly render the math equations below. Please cross-reference them with the part A webpage to make sure that you're looking at the fully correct equation.

A key part of diffusion is the forward process, which takes a clean image and adds noise to it. In this part, we will write a function to implement this. The forward process is defined by:

4.0.1 Delivarables

- Implement the `im_noisy = forward(im, t)` function
- Show the test image at noise level [250, 500, 750]

```

[ ]: def forward(image, t):
    alphas_cumprod = stage_1.scheduler.alphas_cumprod
    alpha_cumprod = alphas_cumprod[t]
    im_noisy = torch.sqrt(1-alpha_cumprod) * torch.randn_like(image) + \
        torch.sqrt(alpha_cumprod) * image
    return im_noisy
blur_image = []

```

```

with torch.no_grad():
    for t in [250, 500, 750]:
        # Get alpha bar
        alpha_cumprod = alphas_cumprod[t]

        # Run forward process
        # ===== your code here! =====

        # TODO:
        # create `im_noisy`, which is `test_im` passed through the forward process
        im_noisy = forward(test_im, t)
        blur_image.append(im_noisy)
        clean_img = im_noisy[0].permute(1,2,0).detach().numpy()
        media.show_image(clean_img / 2. + 0.5)

```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

5 1.2 Classical Denoising

Let's try to denoise these images using classical methods. Again, take noisy images for timesteps [250, 500, 750], but use **Gaussian blur filtering** to try to remove the noise. Getting good results should be quite difficult, if not impossible.

5.0.1 Deliverables

- For each of the 3 noisy test images from the previous part, show your best Gaussian-denoised version side by side.

5.0.2 Hint

- `torchvision.transforms.functional.gaussian_blur` is useful. Here is the [documentation](#).

```

[ ]: import torchvision
for im_noisy in blur_image:
    im_denoised = torchvision.transforms.functional.gaussian_blur(im_noisy,
↪ kernel_size=3, sigma=1)
    im_denoised = im_denoised[0].permute(1, 2, 0).cpu().detach().numpy()
    media.show_image(im_denoised / 2. + 0.5)

```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

6 1.3 Implementing One Step Denoising

Now, we'll use a pretrained diffusion model to denoise. The actual denoiser can be found at `stage_1.unet`. This is a UNet that has already been trained on a *very, very* large dataset of (x_0, x_t) pairs of images. We can use it to recover Gaussian noise from the image. Then, we can remove this noise to recover (something close to) the original image. Note: this UNet is conditioned on the amount of Gaussian noise by taking timestep t as additional input.

Because this diffusion model was trained with text conditioning, we also need a text prompt embedding. We provide the embedding for the prompt "a high quality photo" for you to use. Later on, you can use your own text prompts.

6.0.1 Deliverables

For the 3 noisy images from 1.2 ($t = [250, 500, 750]$):

- Using the UNet, denoise the image by estimating the noise.
- Estimate the noise in the new noisy image, by passing it through `stage_1.unet`
- Remove the noise from the noisy image to obtain an estimate of the original image.
- Visualize the original image, the noisy image, and the estimate of the original image

6.0.2 Hints

- When removing the noise, you can't simply subtract the noise estimate. Recall that in equation 2 we need to scale the noise. Look at equation 2 to figure out how we predict x_0 from x_t and t .
- You will probably have to wrangle tensors to the correct device and into the correct data types. The functions `.to(device)` and `.half()` will be useful. The denoiser is loaded as `half` precision (to save memory), so inputs to the denoiser will also need to be `half` precision.
- The signature for the unet is `stage_1.unet(image, t, encoder_hidden_states=prompt_embeds, return_dict=False)`. You need to pass in the noisy image, the timestep, and the prompt embeddings. The `return_dict` argument just makes the output nicer.
- The unet will output a tensor of shape (1, 6, 64, 64). This is because DeepFloyd was trained to predict the noise as well as variance of the noise. The first 3 channels is the noise estimate, which you will use. The second 3 channels is the variance estimate which you may ignore for now.
- To save GPU memory, you should wrap all of your code in a `with torch.no_grad():` context. This tells torch not to do automatic differentiation, and saves a considerable amount of memory.

```
[ ]: # Please use this prompt embedding
prompt_embeds = prompt_embeds_dict["a high quality photo"]
def forward(image, t):
    alphas_cumprod = stage_1.scheduler.alphas_cumprod
    alpha_cumprod = alphas_cumprod[t]
    im_noisy = torch.sqrt(1-alpha_cumprod) * torch.randn_like(image) + \
        torch.sqrt(alpha_cumprod) * image
    return im_noisy

with torch.no_grad():
```

```

for t in [250, 500, 750]:
    # Get alpha bar
    alpha_cumprod = alphas_cumprod[t]

    # Run forward process
    # ===== your code here! =====

    # TODO:
    # create `im_noisy`, which is `test_im` passed through the forward process
    im_noisy = forward(test_im, t)

    # ===== end of code =====

    # Estimate noise in noisy image
    noise_est = stage_1.unet(
        im_noisy.half().cuda(),
        t,
        encoder_hidden_states=prompt_embeds,
        return_dict=False
    )[0]

    # Take only first 3 channels, and move result to cpu
    noise_est = noise_est[:, :3].cpu()

    # Remove the noise
    # ===== your code here! =====

    # TODO:
    # create `clean_est`, the estimated clean image
    # also run `.detach().numpy()` on the image so we can display it
    clean_est = (im_noisy - torch.sqrt(1-alpha_cumprod) * noise_est) / torch.
    ↪sqrt(alpha_cumprod)
    clean_est = clean_est[0].permute(1,2,0).detach().numpy()
    media.show_image(clean_est / 2. + 0.5)
    # ===== end of code =====

```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

7 1.4 Implementing Iterative Denoising

In part 1.3, you should see that the denoising UNet does a much better job of projecting the image onto the natural image manifold, but it does get worse as you add more noise. This makes sense, as the problem is much harder with more noise!

But diffusion models are designed to denoise iteratively. In this part we will implement this.

In theory, we could start with noise x_{1000} at timestep $T = 1000$, denoise for one step to get an estimate of x_{999} , and carry on until we get x_0 . But this would require running the diffusion model 1000 times, which is quite slow.

It turns out, we can actually speed things up by skipping steps. The rationale for why this is possible is due to a connection with differential equations. It's a tad complicated, and out of scope for this course, but if you're interested you can check out [this excellent article](#).

To skip steps we can create a list of timesteps that we'll call `strided_timesteps`, which will be much shorter than the full list of 1000 timesteps. `strided_timesteps[0]` will correspond to the noisiest image (and thus the largest t) and `strided_timesteps[-1]` will correspond to a clean image (and thus $t = 0$). One simple way of constructing this list is by introducing a regular stride step (e.g. stride of 30 works well).

On the i th denoising step we are at $t = \text{strided_timesteps}[i]$, and want to get to $t' = \text{strided_timesteps}[i+1]$ (from more noisy to less noisy). To actually do this, we have the following formula:

where: - x_t is your image at timestep t - $x_{t'}$ is your noisy image at timestep t' where $t' < t$ (less noisy) - $\bar{\alpha}_t$ is defined by `alphas_cumprod`, as explained above. - $\alpha_t = \bar{\alpha}_t / \bar{\alpha}_{t'}$ - $\beta_t = 1 - \alpha_t$ - x_0 is our current estimate of the clean image using equation 2 just like in section 1.3

The v_σ is random noise, which in the case of DeepFloyd is also predicted. The process to compute this is not very important for us, so we supply a function, `add_variance`, to do this for you.

You can think of this as a linear interpolation between the signal and noise:

([Image Source](#))

For more information, see equations 6 and 7 of the [DDPM paper](#). Be careful about bars above the alpha! Some have them and some do not.

First create the list `strided_timesteps`. You should start at timestep 990, and take step sizes of size 30 until you arrive at 0. After completing the problem set, feel free to try different “schedules” of timesteps.

Also implement the function `iterative_denoise(image, i_start)`, which takes a noisy image `image`, as well as a starting index `i_start`. The function should denoise an image starting at timestep `timestep[i_start]`, applying the above formula to obtain an image at timestep $t' = \text{timestep}[i_start + 1]$, and repeat iteratively until we arrive at a clean image.

Add noise to the test image `im` to timestep `timestep[10]` and display this image. Then run the `iterative_denoise` function on the noisy image, with `i_start = 10`, to obtain a clean image and display it. Please display every 5th image of the denoising loop. Compare this to the “one-step” denoising method from the previous section, and to gaussian blurring.

7.0.1 Deliverables

Using `i_start = 10`: - Create `strided_timesteps`: a list of monotonically decreasing timesteps, starting at 990, with a stride of 30, eventually reaching 0. Also initialize the timesteps using the function `stage_1.scheduler.set_timesteps(timesteps=strided_timesteps)` - Complete

the `iterative_denoise` function - Show the noisy image every 5th loop of denoising (it should gradually become less noisy) - Show the final predicted clean image, using iterative denoising - Show the predicted clean image using only a single denoising step, as was done in the previous part. This should look much worse. - Show the predicted clean image using gaussian blurring, as was done in part 1.2.

7.0.2 Hints

- Remember, the unet will output a tensor of shape (1, 6, 64, 64). This is because DeepFloyd was trained to predict the noise as well as variance of the noise. The first 3 channels is the noise estimate, which you will use here. The second 3 channels is the variance estimate which you will pass to the `add_variance` function
- Read the documentation for the `add_variance` function to figure out how to use it to add the v_σ to the image.
- Depending on if your final images are torch tensors or numpy arrays, you may need to modify the `show_images` call a bit.

```
[ ]: # Make timesteps. Must be list of ints satisfying:
# - monotonically decreasing
# - ends at 0
# - begins close to or at 999

# Make strided_timesteps
# ===== your code here! =====

# TODO:
# create `strided_timesteps`, a list of timesteps, from 990 to 0 in steps of 30
strided_timesteps = list(range(990, -1, -30))

# ===== end of code =====

stage_1.scheduler.set_timesteps(timesteps=strided_timesteps)    # Need this b/c ↵
↳ variance computation
```

```
[ ]: def add_variance(predicted_variance, t, image):
    '''
    Args:
        predicted_variance : (1, 3, 64, 64) tensor, last three channels of the UNet ↵
        ↳ output
        t: scale tensor indicating timestep
        image : (1, 3, 64, 64) tensor, noisy image

    Returns:
        (1, 3, 64, 64) tensor, image with the correct amount of variance added
    '''

    # Add learned variance
    variance = stage_1.scheduler._get_variance(t, ↵
    ↳ predicted_variance=predicted_variance)
```

```

variance_noise = torch.randn_like(image)
variance = torch.exp(0.5 * variance) * variance_noise
return image + variance

```

```

[ ]: def iterative_denoise(image, i_start, prompt_embeds, timesteps, display=True):
    with torch.no_grad():
        for i in range(i_start, len(timesteps) - 1):
            # Get timesteps
            t = timesteps[i]
            prev_t = timesteps[i+1]

            # Get alphas, betas
            # ===== your code here! =====

            # TODO:
            # get `alpha_cumprod` and `alpha_cumprod_prev` for timestep t from
            ↪ `alphas_cumprod`
            # compute `alpha`
            # compute `beta`
            alpha_cumprod = alphas_cumprod[t]
            alpha_cumprod_prev = alphas_cumprod[prev_t]
            alpha = alpha_cumprod / alpha_cumprod_prev
            beta = 1 - alpha

            # ===== end of code =====

            # Get noise estimate
            model_output = stage_1.unet(
                image,
                t,
                encoder_hidden_states=prompt_embeds,
                return_dict=False
            )[0]

            # Split estimate into noise and variance estimate
            noise_est, predicted_variance = torch.split(model_output, image.shape[1],
            ↪ dim=1)

            # Eq (6) and (7) of DDPM
            # ===== your code here! =====

            # TODO:
            # compute `pred_prev_image`, the DDPM estimate for the image at the
            # next timestep, which is slightly less noisy. Use the equation for
            #  $x_{t'}$  in the notes above.
            x_0 = (image - torch.sqrt(1-alpha_cumprod) * noise_est) / torch.
            ↪ sqrt(alpha_cumprod)

```

```

    x_t = image
    pred_prev_image = (torch.sqrt(alpha_cumprod_prev) * beta * x_0 + torch.
↪sqrt(alpha) * (1 - alpha_cumprod_prev) * x_t) / (1 - alpha_cumprod)
    pred_prev_image = add_variance(predicted_variance, t, pred_prev_image)
    # ==== end of code ====

    image = pred_prev_image

    clean = image.cpu().detach().numpy()

    return clean

# Please use this prompt embedding
prompt_embeds = prompt_embeds_dict["a high quality photo"]

# Add noise
i_start = 10
t = strided_timesteps[i_start]
im_noisy = forward(test_im, t).half().to(device)

# Denoise
clean = iterative_denoise(im_noisy,
                          i_start=i_start,
                          prompt_embeds=prompt_embeds,
                          timesteps=strided_timesteps)

# One step denoise
# ===== your code here! =====

# TODO:
# Compute the one step estimate of the clean image. Feel free to copy and paste
# code from part 1.3. Store the image into `clean_one_step`.

with torch.no_grad():
    noise_est = stage_1.unet(
        im_noisy.half().cuda(),
        t,
        encoder_hidden_states=prompt_embeds,
        return_dict=False
    )[0]
    noise_est = noise_est[:, :3]
    alpha_cumprod = alphas_cumprod[t]
    clean_one_step = (im_noisy - torch.sqrt(1-alpha_cumprod) * noise_est) / torch.
↪sqrt(alpha_cumprod)
    clean_one_step = clean_one_step.cpu().detach().numpy()
    media.show_image(clean_one_step[0].transpose(1, 2, 0))

```



```

# ==== end of code ====

# Gaussian blur denoise
# ===== your code here! =====

# TODO:
# Compute the gaussian blurred noisy image, using kernel_size=5 and sigma=2.
# Feel free to copy code from part 1.2. Store the image as `blur_filtered`
# Show results

with torch.no_grad():
    im_denoised = torchvision.transforms.functional.gaussian_blur(im_noisy,
↪kernel_size=5, sigma=2)
    im_denoised = im_denoised[0].permute(1, 2, 0).cpu().detach().numpy()
    media.show_image(im_denoised / 2. + 0.5)
# ==== end of code ====

```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

8 1.5 Diffusion Model Sampling

In part 1.4, we use the diffusion model to denoise an image. Another thing we can do with the `iterative_denoise` function is to generate images from scratch. We can do this by setting `i_start = 0` and passing in random noise. This effectively denoises pure noise. Please do this, and show 5 results of "a high quality photo".

8.0.1 Deliverables

- Show 5 sampled images

8.0.2 Hints

- Use `torch.randn` to make the noise.
- Make sure you move the tensor to the correct device and correct data type by calling `.half()` and `.to(device)`.
- The quality of the images will not be spectacular, but should be reasonable images. We will fix this in the next section with CFG.

```

[ ]: # Please use this text prompt
prompt_embeds = prompt_embeds_dict["a high quality photo"]

generated = {}
image_size = (3, 64, 64)
def generate_images():
    # Add noise
    i_start = 0

```

```

t = strided_timesteps[i_start]
im_noisy = torch.randn(1, *image_size).half().to(device)
# Denoise
clean = iterative_denoise(im_noisy,
                          i_start=i_start,
                          prompt_embeds=prompt_embeds,
                          timesteps=strided_timesteps)

return clean

for i in range(5):
    generated[i] = generate_images()

for i in range(5):
    media.show_image(generated[i][0].transpose(1, 2, 0))

```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

9 1.6 Classifier Free Guidance

You may have noticed that some of the generated images in the prior section are not very good. In order to greatly improve image quality (at the expense of image diversity), we can use a technique called [Classifier-Free Guidance](#).

In CFG, we compute both a noise estimate conditioned on a text prompt, and an unconditional noise estimate. We denote these ϵ_c and ϵ_u . Then, we let our new noise estimate be

$$\epsilon = \epsilon_u + \gamma(\epsilon_c - \epsilon_u) \quad (4)$$

where γ controls the strength of CFG. Notice that for $\gamma = 0$, we get an unconditional noise estimate, and for $\gamma = 1$ we get the conditional noise estimate. The magic happens when $\gamma > 1$. In this case, we get much higher quality images. Why this happens is still up to vigorous debate. For more information on CFG, you can check out [this blog post](#).

Please implement the `iterative_denoise_cfg` function, identical to the `iterative_denoise` function but using classifier-free guidance. To get an unconditional noise estimate, we can just pass an empty prompt embedding to the diffusion model (the model was trained to predict an unconditional noise estimate when given an empty text prompt).

9.0.1 Disclaimer

Before, we used "a high quality photo" as a "null" condition. Now, we will use the actual "" null prompt for unconditional guidance for CFG. In the later part, you should always use "" null prompt for unconditional guidance and use "a high quality photo" for unconditional generation.

9.0.2 Deliverables

- Implement the `iterative_denoise_cfg` function
- Show 5 images of "a high quality photo" with a CFG scale of $\gamma = 7$

9.0.3 Hints

- You will need to run the UNet twice, once for the conditional prompt embedding, and once for the unconditional
- The UNet will predict both a conditional and an unconditional variance. Just use the conditional variance with the `add_variance` function.
- The resulting images should be much better than those in the prior section

```
[ ]: # The condition prompt embedding
prompt_embeds = prompt_embeds_dict['a high quality photo']

# The unconditional prompt embedding
uncond_prompt_embeds = prompt_embeds_dict['']

def iterative_denoise_cfg(image, i_start, prompt_embeds, uncond_prompt_embeds,
    timesteps, scale=7, display=True):
    with torch.no_grad():
        for i in range(i_start, len(timesteps) - 1):
            # Get timesteps
            t = timesteps[i]
            prev_t = timesteps[i+1]

            # Get alphas, betas
            # ===== your code here! =====

            # TODO:
            # Get `alpha_cumprod`, `alpha_cumprod_prev`, `alpha`, `beta`
            # Feel free to copy code from part 1.4
            alpha_cumprod = alphas_cumprod[t]
            alpha_cumprod_prev = alphas_cumprod[prev_t]
            alpha = alpha_cumprod / alpha_cumprod_prev
            beta = 1 - alpha
            # ===== end of code =====

            # Get cond noise estimate
            model_output = stage_1.unet(
                image,
                t,
                encoder_hidden_states=prompt_embeds,
                return_dict=False
            )[0]
```

```

# Get uncond noise estimate
uncond_model_output = stage_1.unet(
    image,
    t,
    encoder_hidden_states=uncond_prompt_embeds,
    return_dict=False
)[0]

# Split estimate into noise and variance estimate
noise_est, predicted_variance = torch.split(model_output, image.shape[1],
↪dim=1)
uncond_noise_est, _ = torch.split(uncond_model_output, image.shape[1],
↪dim=1)

# Do classifier free guidance
# ===== your code here! =====

# TODO:
# Compute the CFG noise estimate and put it in `model_output`.
# Hint: use `model_output` and `uncond_model_output`. Should only require
# one line of code
noise_est = uncond_noise_est + scale * (noise_est - uncond_noise_est)
# ===== end of code =====

# Eq (6) and (7) of DDPM
# ===== your code here! =====

# TODO:
# Get `pred_prev_image`, the next less noisy image.
# Feel free to copy code from part 1.4
# Show denoised image

x_0 = (image - torch.sqrt(1-alpha_cumprod) * noise_est) / torch.
↪sqrt(alpha_cumprod)
x_t = image
pred_prev_image = (torch.sqrt(alpha_cumprod_prev) * beta * x_0 + torch.
↪sqrt(alpha) * (1 - alpha_cumprod_prev) * x_t) / (1 - alpha_cumprod)
pred_prev_image = add_variance(predicted_variance, t, pred_prev_image)
# ===== end of code =====

image = pred_prev_image

clean = image.cpu().detach().numpy()

return clean

```

generated = {}

```

image_size = (3, 64, 64)
def generate_images():
    # Add noise
    i_start = 0
    t = strided_timesteps[i_start]
    im_noisy = torch.randn(1, *image_size).half().to(device)
    # Denoise
    clean = iterative_denoise_cfg(im_noisy,
                                i_start=i_start,
                                prompt_embeds=prompt_embeds,
                                uncond_prompt_embeds=uncond_prompt_embeds,
                                timesteps=strided_timesteps)

    return clean

for i in range(5):
    generated[i] = generate_images()

for i in range(5):
    media.show_image(generated[i][0].transpose(1, 2, 0))

```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

10 1.7 Image-to-image Translation

In part 1.4, we take a real image, add noise to it, and then denoise. This effectively allows us to make edits to existing images. The more noise we add, the larger the edit will be. This works because in order to denoise an image, the diffusion model must to some extent “hallucinate” new things – the model has to be “creative.” Another way to think about it is that the denoising process “forces” a noisy image back onto the manifold of natural images.

Here, we’re going to take the original test image, noise it a little, and force it back onto the image manifold without any conditioning. Effectively, we’re going to get an image that is similar to the test image (with a low-enough noise level). This follows the [SDEdit](#) algorithm.

To start, please run the forward process to get a noisy test image, and then run the `iterative_denoise_cfg` function using a starting index of [1, 3, 5, 7, 10, 20] steps and show the results, labeled with the starting index. You should see a series of “edits” to the original image, gradually matching the original image closer and closer.

10.0.1 Deliverables

- Edits of the test image, using the given prompt at noise levels [1, 3, 5, 7, 10, 20] with text prompt "a high quality photo"

- Edits of 2 of your own test images, using the same procedure.

10.0.2 Hints

- You should have a range of images, gradually looking more like the original image

```
[ ]: # Please use this prompt, as an "unconditional" text prompt
prompt_embeds = prompt_embeds_dict["a high quality photo"]

imgs_dict = {}
start_indexes = [1, 3, 5, 7, 10, 20]
for indexes in start_indexes:
    noise_im = forward(test_im, indexes).half().to(device)
    clean = iterative_denoise_cfg(noise_im,
                                i_start=indexes,
                                prompt_embeds=prompt_embeds_dict['a photo of a
↪man'],
                                uncond_prompt_embeds=prompt_embeds,
                                timesteps=strided_timesteps)
    imgs_dict[indexes] = clean[0].transpose(1,2,0) / 2. + 0.5
for i in start_indexes:
    media.show_image(imgs_dict[i])
```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

10.1 1.7.1 Editing Hand-Drawn and Web Images

This procedure works particularly well if we start with a nonrealistic image (e.g. painting, a sketch, some scribbles) and project it onto the natural image manifold.

Please experiment by starting with hand-drawn or other non-realistic images and see how you can get them onto the natural image manifold in fun ways.

We provide you with 2 ways to provide inputs to the model:

1. download images from the web
2. draw your own images

Please find an image from the internet and apply edits exactly as above. And also draw your own images, and apply edits exactly as above. Feel free to copy the prior cell here. For drawing inspiration, you can check out the examples on [this project page](#).

10.1.1 Deliverables

- 1 image from the web of your choice, edited using the above method for noise levels [1, 3, 5, 7, 10, 20] (and whatever additional noise levels you want)
- 2 hand drawn images, edited using the above method for noise levels [1, 3, 5, 7, 10, 20] (and whatever additional noise levels you want)

10.1.2 Hints

- We provide you with preprocessing code to convert web images to the format expected by DeepFloyd.
- Unfortunately, the drawing interface is hardcoded to be 300x600 pixels, but we need a square image. The code will center crop, so just draw in the middle of the canvas.

```
[ ]: # @title Function to Process Images

def process_pil_img(img):
    '''
    Transform a PIL image
    '''

    # Convert to RGB
    img = img.convert('RGB')

    # Define the transform to resize, convert to tensor, and normalize to [-1, 1]
    transform = transforms.Compose([
        transforms.Resize(64),          # Resize shortest side to 64
        transforms.CenterCrop(64),      # Center crop
        transforms.ToTensor(),          # Convert image to PyTorch tensor
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)) # Normalize to
    ])

    # Apply the transformations and add batch dim
    img = transform(img)[None]

    # Show image
    print("Processed image")
    media.show_image(img[0].permute(1,2,0) / 2 + 0.5)

    return img
```

```
[ ]: # @title Download Images from Web

#####
## CHANGE URL ##
#####
```

```

url = "https://i.pinimg.com/originals/76/e5/d5/76e5d55d0c8c6ec65135b42a2c5cbd98.
↳jpg"
#####
#####

# Download image from URL and process
response = requests.get(url)
web_im = Image.open(BytesIO(response.content))
web_im = process_pil_im(web_im)

```

Processed image

<IPython.core.display.HTML object>

```

[ ]: prompt_embeds = prompt_embeds_dict["a high quality photo"]

imgs_dict = {}
for i_start in tqdm([1, 3, 5, 7, 10, 20]):
    # Add noise
    t = strided_timesteps[i_start]
    im_noisy = forward(web_im, t).half().to(device)

    # Denoise
    clean = iterative_denoise_cfg(im_noisy,
                                i_start=i_start,
                                prompt_embeds=prompt_embeds,
                                uncond_prompt_embeds=uncond_prompt_embeds,
                                timesteps=strided_timesteps,
                                display=False)

    # Add to dict to display later
    imgs_dict[i_start] = clean[0].transpose(1,2,0) / 2. + 0.5

media.show_images(imgs_dict)

```

100%| | 6/6 [00:31<00:00, 5.18s/it]

<IPython.core.display.HTML object>

```

[ ]: # @title Hand Drawn Images

#####
### N.B. ###
#####

# This code is from: https://gist.github.com/karim23657/
↳5ad5e067c1684dbc76c93bd88bf6fa53
# The board is hardcoded to be size 300 x 600, but we're using square images
# so please just draw in the center of the board

```



```

# In addition, this gist may be taken down or change location,
# which will break things in the future

#####
#####

from base64 import b64decode
from IPython.display import HTML
from google.colab.output import eval_js
import urllib.request
board_html = urllib.request.urlopen('https://gist.githubusercontent.com/
↳karim23657/5ad5e067c1684dbc76c93bd88bf6fa53/raw/
↳2ef57f881bc700c2c346bd6c7a7f2d5364b21048/drawing%2520board.html').read().
↳decode('utf-8')

def draw(filename='drawing.png'):
    display(HTML(board_html))
    data = eval_js('triggerImageToServer')
    binary = b64decode(data.split(',')[1])
    with open(filename, 'wb') as f:
        f.write(binary)
    return Image.open(filename).convert('RGB')

drawn_im = draw('myImage.png').resize((64,64))
drawn_im = process_pil_im(drawn_im)

```

<IPython.core.display.HTML object>

Processed image

<IPython.core.display.HTML object>

```

[ ]: prompt_embeds = prompt_embeds_dict["a high quality photo"]

imgs_dict = {}
for i_start in tqdm([1, 3, 5, 7, 10, 20]):
    # Add noise
    t = strided_timesteps[i_start]
    im_noisy = forward(drawn_im, t).half().to(device)

    # Denoise
    clean = iterative_denoise_cfg(im_noisy,
                                  i_start=i_start,
                                  prompt_embeds=prompt_embeds,
                                  uncond_prompt_embeds=uncond_prompt_embeds,
                                  timesteps=strided_timesteps,
                                  display=False)

```

```

# Add to dict to display later
imgs_dict[i_start] = clean[0].transpose(1,2,0) / 2. + 0.5

media.show_images(imgs_dict)

```

100%| | 6/6 [00:25<00:00, 4.31s/it]

<IPython.core.display.HTML object>

10.2 1.7.2 Inpainting (Bonus)

Implement an `inpaint()` function using a binary mask. Inpaint the test Campanile image. Apply inpainting to 2 additional images of your choice. (Hint: Adapt `iterative_denoise_cfg()` with mask updates each step.)

```

[ ]: prompt_embeds = prompt_embeds_dict["a high quality photo"]

# For stage 1: Resize to (64, 64), convert to tensor, rescale to [-1, 1], and
# add a batch dimension. The result is a (1, 3, 64, 64) tensor
# test_im = Image.open('campanile.jpg')
!pip install ultralytics
from ultralytics import SAM
# Load a model
model = SAM("sam2.1_b.pt")
results = model('campanile.jpg', points=[300, 393], labels=[1])
print('Test Mask')
results[0].plot()
# # Show test image
# print('Test image:')
# media.show_image(test_im[0].permute(1,2,0) / 2. + 0.5)

```

Requirement already satisfied: ultralytics in /usr/local/lib/python3.12/dist-packages (8.3.234)

Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.0.2)

Requirement already satisfied: matplotlib>=3.3.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (3.10.0)

Requirement already satisfied: opencv-python>=4.6.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (4.12.0.88)

Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (11.3.0)

Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (6.0.3)

Requirement already satisfied: requests>=2.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.32.4)

Requirement already satisfied: scipy>=1.4.1 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (1.16.3)

Requirement already satisfied: torch>=1.8.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.9.0+cu126)

Requirement already satisfied: torchvision>=0.9.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (0.24.0+cu126)

Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (5.9.5)

Requirement already satisfied: polars>=0.20.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (1.31.0)

Requirement already satisfied: ultralytics-thop>=2.0.18 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.0.18)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (1.3.3)

Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (4.60.1)

Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (1.4.9)

Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (25.0)

Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (3.2.5)

Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (2.9.0.post0)

Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (3.4.4)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (3.11)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (2.5.0)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (2025.11.12)

Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (3.20.0)

Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (4.15.0)

Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (75.2.0)

Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (1.14.0)

Requirement already satisfied: networkx>=2.5.1 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (3.6)

Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages
 (from torch>=1.8.0->ultralytics) (3.1.6)

Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-
 packages (from torch>=1.8.0->ultralytics) (2025.3.0)

Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.6.77 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (12.6.77)

Requirement already satisfied: nvidia-cuda-runtime-cu12==12.6.77 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (12.6.77)

Requirement already satisfied: nvidia-cuda-cupti-cu12==12.6.80 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (12.6.80)

Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (9.10.2.21)

Requirement already satisfied: nvidia-cublas-cu12==12.6.4.1 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (12.6.4.1)

Requirement already satisfied: nvidia-cufft-cu12==11.3.0.4 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (11.3.0.4)

Requirement already satisfied: nvidia-curand-cu12==10.3.7.77 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (10.3.7.77)

Requirement already satisfied: nvidia-cusolver-cu12==11.7.1.2 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (11.7.1.2)

Requirement already satisfied: nvidia-cusparse-cu12==12.5.4.2 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (12.5.4.2)

Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (0.7.1)

Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (2.27.5)

Requirement already satisfied: nvidia-nvshmem-cu12==3.3.20 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (3.3.20)

Requirement already satisfied: nvidia-nvtx-cu12==12.6.77 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (12.6.77)

Requirement already satisfied: nvidia-nvjitlink-cu12==12.6.85 in
 /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
 (12.6.85)

Requirement already satisfied: nvidia-cufile-cu12==1.11.1.6 in

```

/usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralalytics)
(1.11.1.6)
Requirement already satisfied: triton==3.5.0 in /usr/local/lib/python3.12/dist-
packages (from torch>=1.8.0->ultralalytics) (3.5.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-
packages (from python-dateutil>=2.7->matplotlib>=3.3.0->ultralalytics) (1.17.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.12/dist-packages (from
sympy>=1.13.3->torch>=1.8.0->ultralalytics) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.12/dist-packages (from jinja2->torch>=1.8.0->ultralalytics)
(3.0.3)

```

```

image 1/1 /content/campanile.jpg: 1024x1024 1 0, 324.9ms
Speed: 12.9ms preprocess, 324.9ms inference, 0.9ms postprocess per image at
shape (1, 3, 1024, 1024)
Test Mask

```

```

[ ]: array([[202, 115, 48],
           [202, 115, 48],
           [202, 115, 49],
           ...,
           [188, 105, 50],
           [189, 106, 51],
           [189, 106, 51]],

           [[201, 113, 49],
           [201, 113, 49],
           [201, 112, 51],
           ...,
           [188, 105, 50],
           [189, 106, 51],
           [189, 106, 51]],

           [[201, 111, 54],
           [201, 111, 54],
           [201, 111, 54],
           ...,
           [188, 104, 52],
           [189, 105, 53],
           [189, 105, 53]],

           ...,

           [[ 2, 28, 10],
           [ 0, 15, 0],
           [ 0, 28, 3],

```

```

...,
[ 0, 3, 10],
[ 7, 15, 22],
[ 10, 18, 25]],

[[ 0, 38, 24],
[ 22, 73, 59],
[ 51, 104, 91],

...,
[ 0, 25, 22],
[ 11, 38, 35],
[ 1, 28, 24]],

[[111, 158, 149],
[156, 206, 196],
[174, 225, 217],

...,
[ 0, 26, 23],
[ 13, 40, 37],
[ 1, 28, 24]]], dtype=uint8)

```

```

[ ]: def iterative_denoise_cfg_mask(image, i_start, prompt_embeds,
↳uncond_prompt_embeds, timesteps, orig_image, mask, scale=7, display=True):
    # assert mask.shape == image.shape, "Mask must be same shape as image, but
    ↳got mask shape {} and image shape {}".format(mask.shape, image.shape)
    with torch.no_grad():
        for i in range(i_start, len(timesteps) - 1):
            # Get timesteps
            t = timesteps[i]
            prev_t = timesteps[i+1]

            # Get alphas, betas
            # ===== your code here! =====

            # TODO:
            # Get `alpha_cumprod`, `alpha_cumprod_prev`, `alpha`, `beta`
            # Feel free to copy code from part 1.4
            alpha_cumprod = alphas_cumprod[t]
            alpha_cumprod_prev = alphas_cumprod[prev_t]
            alpha = alpha_cumprod / alpha_cumprod_prev
            beta = 1 - alpha
            # ===== end of code =====

            # Get cond noise estimate
            model_output = stage_1.unet(
                image,
                t,

```

```

        encoder_hidden_states=prompt_embeds,
        return_dict=False
    )[0]

    # Get uncond noise estimate
    uncond_model_output = stage_1.unet(
        image,
        t,
        encoder_hidden_states=uncond_prompt_embeds,
        return_dict=False
    )[0]

    # Split estimate into noise and variance estimate
    noise_est, predicted_variance = torch.split(model_output, image.shape[1],
    ↪dim=1)
    uncond_noise_est, _ = torch.split(uncond_model_output, image.shape[1],
    ↪dim=1)

    # Do classifier free guidance
    # ===== your code here! =====

    # TODO:
    # Compute the CFG noise estimate and put it in `model_output`.
    # Hint: use `model_output` and `uncond_model_output`. Should only require
    # one line of code
    noise_est = uncond_noise_est + scale * (noise_est - uncond_noise_est)
    # ===== end of code =====

    # Eq (6) and (7) of DDPM
    # ===== your code here! =====

    # TODO:
    # Get `pred_prev_image`, the next less noisy image.
    # Feel free to copy code from part 1.4
    # Show denoised image

    x_0 = (image - torch.sqrt(1-alpha_cumprod) * noise_est) / torch.
    ↪sqrt(alpha_cumprod)
    x_t = image
    pred_prev_image = (torch.sqrt(alpha_cumprod_prev) * beta * x_0 + torch.
    ↪sqrt(alpha) * (1 - alpha_cumprod_prev) * x_t) / (1 - alpha_cumprod)
    pred_prev_image = add_variance(predicted_variance, t, pred_prev_image)
    # ===== end of code =====

    # Inpaint
    noisy_orig_prev = forward(orig_image, prev_t)
    pred_prev_image = (1.0 - mask) * noisy_orig_prev + mask * pred_prev_image

```

```

        image = pred_prev_image
        clean = image.cpu().detach().numpy()

    return clean
def impaint(img, mask, prompt_embeds, uncond_prompt_embeds):
    # Add noise
    img_size = img.shape
    i_start = 20 # [1, 3, 5, 7, 10, 20]
    t = strided_timesteps[i_start]
    im_noisy = forward(img, t).to(device)
    mask = mask.to(device)
    img = img.to(device)
    im_noisy = (1 - mask) * img + mask * im_noisy
    # Denoise
    clean = iterative_denoise_cfg_mask(im_noisy,
                                      i_start=i_start,
                                      prompt_embeds=prompt_embeds,
                                      uncond_prompt_embeds=uncond_prompt_embeds,
                                      timesteps=strided_timesteps,
                                      orig_image=img,
                                      mask=mask)

    return clean
import torch.nn.functional as F
def get_resize_img_mask(img, mask, size=(64, 64)):
    img = F.interpolate(
        img.float(),
        size=size,
        mode='bilinear',
        align_corners=False
    )

    mask = F.interpolate(
        mask.float(),
        size=size,
        mode='nearest',
    )
    return img, mask

test_im = Image.open('campanile.jpg')
test_im = TF.to_tensor(test_im)
test_im = 2 * test_im - 1
test_im = test_im[None]
mask = results[0].masks.data.float().cpu()
mask_o = mask[None]

img_64, mask_64 = get_resize_img_mask(test_im, mask_o, size=(64,64))

```



```

img, mask, prompt_embeds, uncond_prompt_embeds = img_64.half().to(device),  

↳mask_64.half().to(device), prompt_embeds_dict["a rocket ship"],  

↳prompt_embeds_dict['']  

generated= inpaint(img, mask, prompt_embeds, uncond_prompt_embeds)

img_256, mask_256 = get_resize_img_mask(test_im, mask_o, size=(256, 256))  

img_256, mask_256 = img_256.half().to(device), img_256.half().to(device)

generated_256 = stage_2(  

image=torch.tensor(generated).to(device),  

num_inference_steps=20,  

prompt_embeds=prompt_embeds,  

negative_prompt_embeds=uncond_prompt_embeds,  

output_type="pt",  

).images

generated_256 = (1.0 - mask_256) * img_256 + mask_256 * generated_256.to(device)  

generated_256 = generated_256.cpu().detach().numpy()

media.show_image(generated[0].transpose(1,2,0) / 2. + 0.5)  

media.show_image(generated_256[0].transpose(1,2,0) / 2. + 0.5)

```

```
0%|          | 0/20 [00:00<?, ?it/s]
```

```
<IPython.core.display.HTML object>
```

```
<IPython.core.display.HTML object>
```

10.3 1.8 Visual Anagrams

In this part, we aim to implement [Visual Anagrams](#) and create optical illusions with diffusion models. In this part, we will create an image that looks like "an oil painting of an old man", but when flipped upside down will reveal "an oil painting of people around a campfire".

To do this, we will denoise an image x_t at step t normally with the prompt "an oil painting of an old man", to obtain noise estimate ϵ_1 . But at the same time, we will flip x_t upside down, and denoise with the prompt "an oil painting of people around a campfire", to get noise estimate ϵ_2 . We can flip ϵ_2 back, to make it right-side up, and average the two noise estimates. We can then perform a reverse diffusion step with the averaged noise estimate.

The full algorithm is:

```

$ _1 = UNet(x_t, t, p_1) $
$ _2 = flip(UNet(flip(x_t), t, p_2))$
$ = (_1 + _2) / 2 $

```

where UNet is the diffusion model UNet from before, flip($\cdot$) is a function that flips the image, and p_1 and p_2 are two different text prompt embeddings. And our final noise estimate is ϵ . Please

implement the above algorithm and show example of an illusion.

10.3.1 Deliverables

- Correctly implemented `visual_anagrams` function
- A visual anagram where on one orientation "an oil painting of people around a campfire" is displayed and, when flipped, "an oil painting of an old man" is displayed.
- 2 more illusions of your choice that change appearance when you flip it upside down (see the appendix to generate your own prompts)

10.3.2 Hints

- You may have to run multiple times to get a really good result for the same reasons as above

```
[ ]: strided_timesteps = list(range(990, -1, -1))
import tqdm

stage_1.scheduler.set_timesteps(timesteps=strided_timesteps)

def get_noise_varience(image, prompt, uncond_prompt, t, scale):
    model_output = stage_1.unet(
        image,
        t,
        encoder_hidden_states=prompt,
        return_dict=False
    )[0]
    uncond_model_output = stage_1.unet(
        image,
        t,
        encoder_hidden_states=uncond_prompt,
        return_dict=False
    )[0]
    noise_est, predicted_variance = torch.split(model_output, image.shape[1],
    ↪dim=1)
    uncond_noise_est, _ = torch.split(uncond_model_output, image.shape[1], dim=1)
    noise_est = uncond_noise_est + scale * (noise_est - uncond_noise_est)
    return noise_est, predicted_variance
def make_flip_illusion(image, i_start, prompt_embeds, uncond_prompt_embeds,
    ↪timesteps, scale=7, display=True):
    dim = 2
    prompt_1 = prompt_embeds[0]
    prompt_2 = prompt_embeds[1]
    with torch.no_grad():
        for i in tqdm.tqdm(range(i_start, len(timesteps) - 1)):
            t = timesteps[i]
            prev_t = timesteps[i+1]

            alpha_cumprod = stage_1.scheduler.alphas_cumprod[t]
```

```

alpha_cumprod_prev = stage_1.scheduler.alphas_cumprod[prev_t]
alpha = alpha_cumprod / alpha_cumprod_prev
beta = 1 - alpha

flip_image = torch.flip(image, [dim])

noise_est, predicted_variance = get_noise_variance(image, prompt_1,
↳uncond_prompt_embeds, t, scale)
flip_noise_est, flip_predicted_variance = get_noise_variance(flip_image,
↳prompt_2, uncond_prompt_embeds, t, scale)

flip_noise_est, flip_predicted_variance = torch.flip(flip_noise_est,
↳[dim]), torch.flip(flip_predicted_variance, [dim])

noise_est = (noise_est + flip_noise_est) / 2
predicted_variance = (predicted_variance + flip_predicted_variance) / 2

x_0 = (image - torch.sqrt(1-alpha_cumprod) * noise_est) / torch.
↳sqrt(alpha_cumprod)
x_t = image
pred_prev_image = (torch.sqrt(alpha_cumprod_prev) * beta * x_0 + torch.
↳sqrt(alpha) * (1 - alpha_cumprod_prev) * x_t) / (1 - alpha_cumprod)
pred_prev_image = add_variance(predicted_variance, t, pred_prev_image)

image = pred_prev_image
clean = image.cpu().detach().numpy()

return clean
prompt_embeds = [prompt_embeds_dict["an oil painting of an old man"],
↳prompt_embeds_dict["an oil painting of people around a campfire"]]
uncond_prompt_embeds = prompt_embeds_dict['']

image_size = (3, 64, 64)

# Add noise
i_start = 0
im_noisy = torch.randn(1, *image_size).half().to(device)
clean_flip_1 = make_flip_illusion(im_noisy,
                                i_start=i_start,
                                prompt_embeds=prompt_embeds,
                                uncond_prompt_embeds=uncond_prompt_embeds,
                                timesteps=strided_timesteps)

prompt_embeds = [prompt_embeds_dict["a lithograph of waterfalls"],
↳prompt_embeds_dict["an oil painting of a snowy mountain village"]]

```

```

im_noisy = torch.randn(1, *image_size).half().to(device)
clean_flip_2 = make_flip_illusion(im_noisy,
                                i_start=i_start,
                                prompt_embeds=prompt_embeds,
                                uncond_prompt_embeds=uncond_prompt_embeds,
                                timesteps=strided_timesteps)

prompt_embeds = [prompt_embeds_dict["a rocket ship"], prompt_embeds_dict["a
↳lithograph of a skull"]]
im_noisy = torch.randn(1, *image_size).half().to(device)
clean_flip_3 = make_flip_illusion(im_noisy,
                                i_start=i_start,
                                prompt_embeds=prompt_embeds,
                                uncond_prompt_embeds=uncond_prompt_embeds,
                                timesteps=strided_timesteps)

# ['an oil painting of a snowy mountain village',
#  'a photo of the amalfi cost',
#  'a photo of a man',
#  'a photo of a hipster barista',
#  'a photo of a dog',
#  'an oil painting of people around a campfire',
#  'an oil painting of an old man',
#  'a lithograph of waterfalls',
#  'a lithograph of a skull',
#  'a man wearing a hat',
#  'a high quality photo',
#  '',
#  'a rocket ship',
#  'a pencil']
media.show_image(clean_flip_1[0].transpose(1,2,0) / 2. + 0.5)
media.show_image(clean_flip_2[0].transpose(1,2,0) / 2. + 0.5)
media.show_image(clean_flip_3[0].transpose(1,2,0) / 2. + 0.5)

```

```

100%|      | 990/990 [05:26<00:00, 3.03it/s]
100%|      | 990/990 [05:26<00:00, 3.03it/s]
100%|      | 990/990 [05:27<00:00, 3.02it/s]

```

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>

<IPython.core.display.HTML object>